

Vigil: Trust and Risk Intelligence for Mobile Money

AI Dev Fest 2026 AI Hackathon, Track 01

Team JU_MergeConflict
Daffodil International University

October 2026

1 Problem

Mobile money runs on trust. In Bangladesh, wallets like upay move everyday money for millions of people, and one scam call or one hijacked account can wipe out a user's savings along with their faith in digital payments. Fraud analysts fight this daily, but they drown in alerts. A typical system shows a bare risk score with no explanation, so every serious case costs a long manual investigation. Slow reviews mean stolen money keeps moving, analysts burn out, and users leave the platform.

2 Proposed idea

Vigil is an AI investigation console that answers the three questions every fraud case raises:

1. What happened?
2. Why is it risky?
3. What should upay do next?

Each transfer is scored, the score is explained in plain auditable reasons and mapped to a clear action, and the whole case is written up as a short narrative in English or Bangla. High risk never blocks money on its own. It holds the transaction, asks for step up authentication, and puts a human analyst in charge.

3 Implemented solution

Figure 1 shows the full loop, which runs as three parts.

Scoring API (FastAPI). A transfer arrives as sender, receiver, amount, device, location, and timestamp. The API returns a risk score between 0 and 1, a level of Low, Medium, or High, the top three reasons, a recommended action, and a written narrative, in about 25 milliseconds.

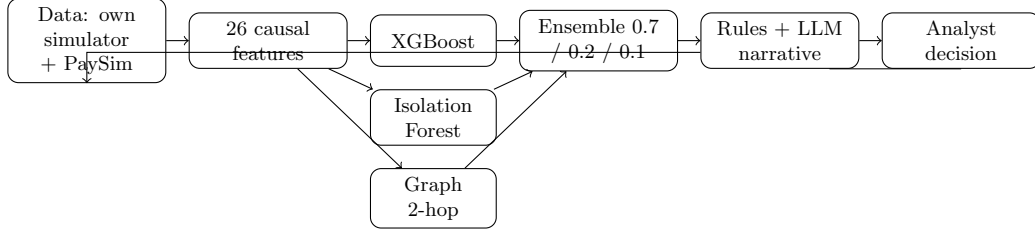


Figure 1: Vigil pipeline, from data to analyst action with a retraining loop.

Analyst console (static web page). A risk sorted queue with level filters and search, a case view with narrative, reasons, component scores, a mule ring panel, and a wallet timeline, plus a scoring playground and decision buttons that log Allow, Step up, or Freeze for future retraining.

Evaluation harness. Measures the system on held out data and reports ranking quality, fairness across districts and account ages, latency, and business impact.

All development used synthetic data we generated ourselves plus public data, following privacy by design. No production customer data was needed or used.

4 Key features

- Real time scoring with auditable top three reasons, backed by per transaction SHAP values.
- English and Bangla narratives with a deterministic offline fallback, so the demo works without internet.
- Mule ring discovery through the transaction graph, with fraud wallets near a receiver listed in each case.
- Strict input validation, isolated scoring traffic that cannot poison anyone’s features, and a 47 test suite covering scoring, validation, security regressions, and the language model path.
- Thresholds in a config file, so new requirements integrate without retraining the models.

5 AI approach

5.1 Data

Table 1 compares our two sources. Our simulator injects scam, takeover, mule, and agent patterns plus realistic noise. PaySim is a public mobile money dataset calibrated on real logs, used only for independent validation, never for training the product.

Source	Transactions	Fraud cases	Key signals
Own simulator	50,000	~2,000	device, location, reset, velocity, graph
PaySim (adapted)	108,213	8,213	amount, velocity, type, graph

Table 1: Training data versus independent validation data.

5.2 Models

XGBoost classifier. Learns fraud shapes from tens of thousands of labeled examples and outputs a probability of fraud.

IsolationForest. Trained only on legitimate traffic, it flags behavior unlike anything normal, with scores calibrated to training percentiles.

Transaction graph. Adds suspicion for receivers sitting near known fraud wallets, catching mule rings invisible to single transfers.

Ensemble. A weighted blend of 0.7, 0.2, and 0.1 turns the three signals into one score, mapped to allow, review, or hold with human oversight.

A grounded language model narrates each case from the computed facts only, under a fixed template with sanitized inputs.

5.3 Evaluation

Table 2 reports the same code with no tuning on both datasets. On our data the system reaches 0.998 AUC against 0.81 for hand written rules. On PaySim, where our best signals are unavailable, it holds 0.90 while the rules collapse to a coin flip. We deliberately refused leakage features that would have printed perfect scores and proved nothing.

Metric	Ours (model)	Ours (rules)	PaySim (model)	PaySim (rules)
AUC	0.9982	0.8119	0.8985	0.5049
Precision@100	1.00	0.86	1.00	0.83
Recall@5%FPR	0.9938	0.2149	0.6286	0.0000
p95 latency	25.9 ms		23.2 ms	

Table 2: Same pipeline, same metrics, foreign data included.

6 Intended real life impact

- **Analysts:** a fifteen minute manual investigation becomes a two minute review, converted in our business simulation into loss prevented in BDT and analyst hours saved per thousand high risk holds.
- **Users:** faster holds on scams and takeovers stop stolen money sooner, and clear warnings protect first time and vulnerable customers before they pay a stranger.
- **upay:** every held scam is trust kept, and trust is what grows a wallet.

The API contract is stateless and ready to plug into a real stream, with weekly retraining on governed labels as the path from prototype to pilot. Responsible by design throughout: synthetic data only, explainable scores, fairness measured, human oversight on every consequential action.